CAS CORE 8: A COMPATIBILITY AND EVIDENCE KERNEL FOR AUDITABLE COGNITIVE-ARCHITECTURE SOFTWARE

LITTLE HENDERSON

Abstract. CAS Core 8 is a small, auditable compatibility and evidence kernel for the Cognitive Architecture System (CAS) software family. It is deliberately not a rewrite of CAS v7.7 and is not installed on any production route. Its thesis is that trustworthy cognitive-architecture software begins with a kernel that cannot be talked into overclaiming: before any v8 semantics can be trusted, source identity, semantic namespace, compatibility profile, registry integrity, action intent, and claim boundaries must be explicit enough to test.

This paper develops the kernel's five mechanisms in full: (1) structural semantic references over disjoint namespaces, with an injective-rendering guarantee and a hard rule that positional or historical correspondences are typed as translations, alignments, or hypotheses—never silently as identities; (2) a research-boundary object whose ring, license, and disclosure fields must be jointly consistent and are derived and rechecked by every public operation; (3) a deterministic canonical evidence encoding—sorted-key UTF-8 JSON; rejection of unordered containers, binary floats, NaN, and infinity; tagged exact decimals; one serialization route per public type; bounded, cycle-aware canonicalization—justified by an explicit threat analysis of digest-collision and type-pun attacks; (4) a hash-verified, process-isolated legacy compiler compatibility profile with an eight-step deterministic-closure protocol, an advisory-lock concurrency discipline, an exact worker-failure taxonomy, and a stated non-sandbox boundary; and (5) exact-byte-backed claim and theorem registries whose loaders reject every structural irregularity and whose records carry deterministic digests bound to the registry bytes. A single design commitment runs through all five: *structural consistency is never authorization.* In v0.1 there is no code path that grants authorization; claim-bearing intent is denied even when a locally constructed ledger says "licensed," because the kernel has no authenticated reviewer or legal trust root. We state the future claim-authorization design constraints, the relation to the four companion candidates, the bounded software evidence, and the complete open-obligation set.

Contents

1. Introduction 2
1.1. Two failure classes 2
1.2. Design inversion 2
1.3. Status 2
2. Semantic identity 2
3. The research boundary object 3

Date: August 7, 2026.

Research paper v0.2, 2026-08-07 (supersedes white paper v0.1). Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. Source basis: `CAS_CORE_V8_CANDIDATE_SPEC_v0.1.md` (SHA-256 `5FE6F91D005C7268D32C3943C2851138B30E2492C7D73877A04B30685CFCA1FC`); implementation `src/cas_core_v8/` (`identity.py`, `registry.py`, `runtime_gate.py`, `semantics.py`, `compatibility.py`, `_compiler_worker.py`); Gate 4 receipt: 61 test methods, suite test-ID SHA-256 `E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`, within an integrated 208-method, 272-subtest run over 54 hash-bound artifacts. Drafting this paper promotes no claim; independent software, formal-methods, semantics, and product-safety review remains open.

4. Canonical evidence encoding 4
5. Legacy compiler compatibility profile 4
6. Registry integrity 5
7. Runtime decision model 5
8. Threat analysis 6
9. Relation to companion candidates 6
10. Bounded software evidence 7
11. Open obligations 7
12. Related work 7
13. Conclusion 7
References 8

1. Introduction

1.1. **Two failure classes.** Long-lived research software in the TKS/CAS program has repeatedly exhibited two failure classes that no amount of downstream testing repairs.

Semantic drift. The program spans multiple ontologies—TKS Foundations and Worlds; CAS Drives, Domains, Registers, and States; Noetic Topos objects—whose elements are related by history and position but are not interchangeable. Prior audits found positional relabels treated as paraphrases and cross-namespace conflations voiding otherwise sound work products. Untyped string identifiers make such drift invisible: nothing in `"F5"` says whether it names a TKS Foundation, a CAS Drive, or a coincidence.

Unauthorized claim promotion. A green test, a self-computed hash, or a generated receipt is structural evidence about software behavior. Nothing in such an artifact establishes reviewer identity, legal authority, or empirical validity. Historical AI-authored, self-hashed "licensing receipts" in the program's own past demonstrated how easily structural artifacts masquerade as authority. A kernel that cannot distinguish *consistent* from *authorized* will eventually launder one into the other.

1.2. **Design inversion.** CAS Core 8 addresses both classes by inversion: instead of building a permissive system and subtracting dangerous behavior, it builds a kernel in which the dangerous behaviors are unrepresentable or fail closed, and everything a future CAS v8 wants to assert—semantics, conformance, claims—must be *added* through separately reviewed gates. Sections 2–7 develop the five mechanisms; Section 8 gives the threat analysis each mechanism answers; Section 9 places the kernel among its companion candidates; Sections 10–11 state the evidence and the open obligations exactly.

1.3. **Status.** Everything here is a Ring-4 research candidate: admission OPEN, product NOT LICENSED, disclosure withheld. The bounded Gate-4 receipts are historical software evidence, not experiments, review, or authority.

2. Semantic identity

Definition 2.1 (Semantic reference)**.** A *semantic reference* is the structural tuple

$$(\text{namespace},\ \text{local_id},\ \text{version},\ \text{source_id},\ \text{definition_sha256}),$$

where the namespace ranges over the disjoint families TKS-FOUNDATION, TKS-WORLD, CAS-DRIVE, CAS-DOMAIN, CAS-REGISTER, CAS-STATE, and NOETIC-TOPOS-OBJECT, and `definition_sha256` is the digest of the exact defining text in the identified source.

Identity is structural equality of the full tuple. Binding the defining text's hash means that a reference does not merely name a concept; it names a concept *as defined by particular bytes*, so silent definitional drift between sources is detectable at the identity layer.

Rule 2.2 (No silent identity)**.** A positional or historical correspondence between references in different namespaces is represented as a typed link with kind TRANSLATION, ALIGNMENT, or HYPOTHESIS. It is never represented as IDENTITY. There is no coercion path from any link kind to identity.

Rule 2.2 encodes at the type level the canon rule that positional relabels—for example, between TKS Foundation positions and CAS Drive names—are not paraphrases. The historical incident motivating it: a prior work product conflated positionally corresponding Drive and Foundation names and was voided in review. Under this kernel that conflation is unrepresentable: the strongest representable statement is a TRANSLATION link carrying its own source and review state.

Invariant 2.3 (Injective rendering)**.** *The rendered string form of a semantic reference is injective on structural tuples: delimiter-containing components cannot render to the same identifier as a different tuple.*

Design argument. Rendering escapes or length-prefixes every component before joining, so the concatenation is uniquely parseable; two distinct tuples therefore differ at the first divergent component in any parse. The property is enforced by construction and regression-tested with delimiter-bearing component fixtures. □

Rule 2.4 (Exact-type identity)**.** Equality-overriding subclasses are not admitted as identity-bearing public values: exact runtime types and primitive fields determine every boundary and semantic identity check.

Without Rule 2.4, a subclass overriding equality could impersonate a reference or boundary object and defeat every downstream check while remaining "equal" to the honest value. The kernel compares types exactly and rejects subclass instances at its public interfaces.

3. The research boundary object

Definition 3.1 (Research boundary)**.** A *research boundary* is the triple (admission state, product state, public disclosure). Ring 4 requires jointly: admission OPEN; product NOT LICENSED; disclosure *false*. Any contradictory combination—for example, OPEN admission with a licensed product state, or disclosure *true* in Ring 4—is invalid at construction.

Invariant 3.2 (Recheck, do not trust)**.** *Every public operation derives and rechecks the boundary from its inputs rather than trusting a shared mutable object or a recomputable self-hash. A structural receipt or green test does not alter a boundary. A self-digest detects accidental or malicious mutation after construction, but is not a signature, reviewer identity, or authority grant.*

The distinction in Invariant 3.2 between *tamper evidence* and *authority* is load-bearing for the whole kernel: a self-digest answers "has this object changed since construction?"; it cannot answer "who authorized this object?"—the latter requires an authenticated trust root that v0.1 deliberately lacks (Section 7).

4. Canonical evidence encoding

Evidence digests are load-bearing: every registry record, compatibility receipt, and replay comparison reduces to hash equality of canonical encodings. Any nondeterminism or type pun in the encoder silently invalidates every downstream comparison. The encoding is therefore specified as a closed rule set.

Rule 4.1 (Canonical encoding)**.** The evidence encoding: (i) is deterministic UTF-8 JSON with sorted string keys; (ii) rejects unordered containers (sets), binary floats, NaN, and infinity; (iii) tags exact decimals so that Decimal(`"1.0"`), the string `"1.0"`, and the integer 1 have pairwise distinct encodings; (iv) gives each public type exactly one declared serialization route; and (v) is bounded and cycle-aware, failing closed on graphs exceeding declared depth or node limits and on any cycle.

Rule 4.2 (Byte identity vs. comparison identity)**.** Raw bytes and the declared CRLF-to-LF comparison identity are recorded separately. Lone-CR, CRLF, and lone-LF counts are never conflated: a file's raw digest and its newline-normalized comparison digest are distinct fields with distinct roles, and neither substitutes for the other.

Rule 4.2 answers a failure the program has already experienced: cross-platform newline rewriting changing raw digests while leaving semantic content intact, and conversely, normalization masking a real byte-level difference. Recording both identities makes each failure visible instead of silent.

Rule 4.3 (Identifier profiles)**.** Review-visible machine identifiers use declared normalized/ASCII profiles; unrestricted prose Unicode is carried in separate fields and never participates in identifier comparison.

5. Legacy compiler compatibility profile

The only admitted legacy profile in v0.1 binds the exact active Python CAS-notation-compiler bytes listed in the Gate 4 record. The profile is explicitly `legacy_only`: it executes the verified legacy compiler for comparison purposes and produces *no* CAS v8 semantic translation.

Protocol 5.1 (Deterministic closure)**.** A compatibility execution proceeds:

(1) verify the requested source hash; read and hash the compiler bytes *before* execution;
(2) force the Python-only engine profile and an exact frozen policy (a separately hash-bound native engine profile may be introduced later, but is not);
(3) reject `use` imports—no import-DAG manifest is yet admitted, so multi-file programs are outside the profile;
(4) execute the verified compiler bytes in a fresh `python -I -S -B` worker with a sanitized environment, rather than inheriting the parent's module state;
(5) sample and bind the worker runtime before compiler code executes: interpreter fields, `sys.platform`/POSIX `os.uname`, and the profile-bound Windows architecture inputs, without WMI or subprocess probing;
(6) serialize cooperating worker executions through a profile-bound advisory lock with a 90-second acquisition bound, 10-millisecond poll interval, and 10-millisecond post-release scheduler yield, without retrying a failed worker and without claiming lock fairness;

(7) record raw and LF compiler identities, interpreter and worker bytes, import search paths, loaded-module inventory, exact syntax/semantic/engine profiles, result digest, diagnostics, and the research boundary; and
(8) require fresh replay against the same source and compiler identity before any structural decision can pass.

Rule 5.2 (Failure taxonomy)**.** Worker transport failures record only exit code, stdout/stderr byte counts, and their SHA-256 digests. An empty or malformed response fails closed and is never retried. If two successful workers disagree on their dependency profiles, the error records both profile hashes and at most 32 keyed difference paths; environment values and dependency paths are represented only by hashes.

The retry prohibition is deliberate: retrying a nondeterministic failure converts flakiness into silent selection bias over receipts. The bounded difference-path list keeps disagreement diagnosable without exfiltrating environment contents into evidence records.

Claim Boundary 5.3 (Isolation is not a sandbox). Protocol 5.1 achieves Python-process isolation, *not* an operating-system sandbox. Host native libraries and the operating system are not content-addressed. Every receipt exposes both limitations as *false* capability fields, so no downstream consumer can honestly read a receipt as hermetic-execution evidence.

Rule 5.4 (Fractal bridge)**.** Legacy fractal digit sequences are bridged to the TKS v8 kernel only through an explicit caller-supplied catalog of typed finite arrows; composition uses semigroupoid paths; missing digits or source/target mismatches fail closed. The bridge proves type-correct composition in the finite reference kernel—never the meaning of a legacy digit.

6. Registry integrity

The claim and theorem registries are exact-byte-backed snapshots validated against the Project Coherence record contracts.

Rule 6.1 (Byte-backed loading)**.** Parsed headers, schemas, and records are reconstructed from and rechecked against the retained raw ledger and schema bytes. A direct constructor or dataclass copy cannot preserve an old raw identity while substituting parsed content: the loader recomputes the parse from bytes and compares. The loader rejects duplicate JSON keys, unknown root or record fields, wrong enums or shapes, invalid identifiers, ambiguous versions, and missing required metadata. Loaded values are recursively immutable, and each selected record carries a deterministic digest bound to the registry bytes.

Invariant 6.2 (Metadata is not authority)**.** *The text* LICENSED *inside a record is necessary metadata, not sufficient authority. The v0.1 candidate has no authenticated reviewer or legal trust root and therefore cannot authorize any claim-bearing action, regardless of registry content.*

7. Runtime decision model

Action operation, target, and effect class are explicit enums; there is no stringly-typed action path. The decision procedure distinguishes *nonclaim research evidence*—which may pass a structural replay comparison when every exact compatibility field and digest validates under Protocol 5.1—from *claim-bearing intent*, which is always denied.

Invariant 7.1 (No authorization path)**.** *In v0.1, the field* `authorization_granted` *and the compatibility spelling* `allowed` *are constant* false*; there is no code path that sets either true.*

Claim-bearing intent is denied even if a locally constructed ledger says LICENSED. *A self-hash or Python-local sentinel is not issuance provenance.*

Rule 7.2 (Future authorization constraints)**.** Any future claim-authorization implementation requires a separately reviewed design binding, at minimum: an exact trusted registry/schema manifest; an authenticated authority and trust root; reviewer and admission receipts; Evidence Packet identities and verification status; the claim ID and exact version; and the code, model, environment, semantic, and calibration identities of the requested operation. Absent all of these, the kernel's answer remains *no*.

8. Threat analysis

Each mechanism above answers specific threats. We state the principal ones and the blocking mechanism.

Threat 8.1 (Namespace conflation). An agent or tool treats a positional correspondence as identity and propagates conclusions across ontologies. *Blocked by* Definition 2.1 and Rule 2.2: the conflation is unrepresentable; the strongest representable statement carries link kind and review state.

Threat 8.2 (Identifier collision). Crafted components render two distinct references to one identifier, aliasing records. *Blocked by* Invariant 2.3.

Threat 8.3 (Equality spoofing). A subclass overrides equality to impersonate a boundary or reference. *Blocked by* Rule 2.4.

Threat 8.4 (Digest instability and type puns). Hash-order iteration, float formatting, or Decimal/string collisions make equal-looking evidence hash differently, or different evidence hash equal. *Blocked by* Rule 4.1; newline rewriting specifically by Rule 4.2.

Threat 8.5 (Compiler-identity drift). A compatibility receipt is produced against different compiler bytes than claimed, or under inherited interpreter state. *Blocked by* Protocol 5.1 steps (1), (4), (5), and the replay requirement (8).

Threat 8.6 (Receipt laundering). A structural receipt, self-hash, or generated "license" is presented as authority. *Blocked by* Invariants 3.2, 6.2, and 7.1: there is no representable authorization in v0.1 to launder into.

Threat 8.7 (Registry substitution). Parsed registry content is swapped while raw identity is retained. *Blocked by* Rule 6.1: parses are recomputed from bytes and compared; constructor-path substitution cannot preserve the byte-bound digest.

9. Relation to companion candidates

CAS Core 8 is the shared substrate of the Gate 4 engineering lane. *CAS PCBA 8* consumes the semantic-identity, tagged-value, canonical-evidence, and research-boundary contracts for its board model, design-rule checks, and traces. *CAS-Ohm 1.0* consumes the same evidence discipline for typed quantities, calibration profiles, and fail-closed research packets, and mirrors the authorization split as its "structural completeness vs. authenticated eligibility" axis. *TKS v8.0* supplies the finite typed-arrow kernel referenced by Rule 5.4. *Noetic Topos 2.0* supplies the ambient-versus-resolved evidence distinction the kernel's records anticipate: a marked cover proves no measurement true, and raw records live in the ambient layer until restriction and gluing are defined.

The dependency direction is strict: companions depend on Core's contracts; Core depends on none of their semantics. This keeps the kernel reviewable in isolation.

10. Bounded software evidence

At the bound Gate 4 receipt (2026-08-04, commit `a1cd9901`): the focused CAS Core v8 suite records 61 passing test methods with suite test-ID SHA-256 `E2AF5014...`, within an integrated three-suite run of 208 methods and 272 parameterized subtests over 54 hash-bound artifacts; 11 claim records all Ring 4 and NOT LICENSED; and 53 review criteria with 20 open. The repository-wide collector was *not* clean at that receipt (four pre-existing package-path namespace collisions after 1,200 collected items), so the focused suites cannot support a repository-wide verification claim.

The suite exercises: reference identity and injective rendering (including delimiter fixtures); boundary consistency and recheck behavior; canonical-encoding acceptance and rejection cases (sets, floats, NaN, infinity, decimal tagging, cycles, depth bounds); compatibility-protocol hash verification, worker sanitization, lock behavior, failure taxonomy, and replay requirement; registry loader rejection cases; and the constant-false authorization invariants.

Claim Boundary 10.1. These numbers are historical, source-bound software receipts. They establish tested software behavior under the exact fixtures and profiles only—not hermetic execution, semantic correctness of CAS content, empirical validity, claim authority, or review.

11. Open obligations

(1) A hermetic subprocess and toolchain bundle is not established by Python-level isolation alone (Boundary 5.3); content-addressing the interpreter, native libraries, and OS surface remains open.
(2) Python/Rust conformance is not established; engine profiles remain separate until a cross-implementation differential-testing campaign exists.
(3) No v8 parser or full semantic migration is claimed; the profile is `legacy_only`.
(4) No claim-authority implementation is admitted; Rule 7.2 states its minimum design constraints.
(5) An import-DAG manifest for multi-file programs is not admitted (Protocol 5.1 step 3).
(6) Independent software, formal-methods, semantics, and product-safety review is required before any status change.

12. Related work

The mechanisms adapt established practice to the CAS setting rather than claiming novelty: canonical JSON encodings for stable digests (cf. RFC 8785 [1]); content-addressed artifact identity and reproducible-build discipline [2]; provenance separation of entity, activity, and derivation [3]; and the principle that authorization requires an authenticated trust root distinct from integrity checking [4]. The kernel's specific contribution is the composition of these into a fail-closed substrate for a research program with documented drift and overclaim incidents, together with the constant-false authorization invariant as a design stance.

13. Conclusion

CAS Core 8 makes a narrow, testable promise: identities are structural, boundaries are rechecked, encodings are deterministic, legacy execution is hash-verified and replayed, registries are byte-backed, and there is no path to "authorized" anywhere in the kernel. The threat analysis of Section 8 is the paper's real argument: each historically observed failure class corresponds to a representable-state restriction or a fail-closed rule, so the failure now

surfaces as a type error, a rejection, or a constant-false field instead of a plausible artifact. Everything a future CAS v8 wants to assert must be added through reviewed gates. That inversion—subtraction replaced by gated addition—is the contribution.

References

[1] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.
[2] C. Lamb and S. Zacchiroli, "Reproducible builds: increasing the integrity of software supply chains," *IEEE Software* 39(2), 2022.
[3] W3C, *PROV-O: The PROV Ontology*, 2013.
[4] R. Anderson, *Security Engineering*, 3rd ed., Wiley, 2020.
[5] Project Coherence, Gate 4 record, review matrix v0.1, and artifact manifest v0.1.
[6] Project Coherence record contracts (claim and theorem registry schemas).